Module 9

RNA-clique

Goal: Learn how to the RNA-clique tool for calculating genetic distances.

Input(s):
```
SRR2321388/transcripts.fasta
SRR2321385/transcripts.fasta
SRR8003761/transcripts.fasta
SRR8003762/transcripts.fasta
SRR7990321/transcripts.fasta
SRR8003736/transcripts.fasta
tall_fescue_metadata.csv
```

Output(s):
```
graph.pkl
distance_matrix.h5
heatmap.svg
tree.svg
pcoa.svg
```

9.1 Genetic distances based on RNA-Seq data

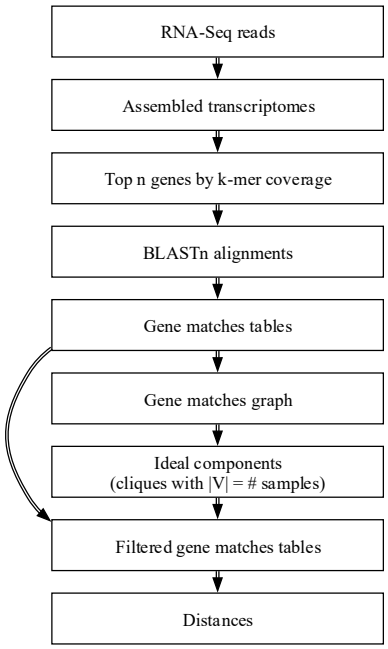
This exercise will have you obtain a **pairwise genetic distance matrix** quantifying differences in the genomes of a set of six individual tall fescue (*Lolium arundicaeum*) plants (six "samples") for which we have RNA-Seq reads. We will use a piece of software called **RNA-clique** to compute such distances from sets of transcripts (**transcriptomes**) we have already assembled from the RNA-Seq reads for each sample. Although RNA-Seq data are typically used in studies of gene expression, RNA-clique offers a cost-effective way to reuse the same data for a different purpose.

In our case, the inputs to RNA-clique will be assembled transcriptomes for the six tall fescue plants of interest. RNA-clique is designed to be used with transcriptomes assembled with rnaSPAdes, the same *de novo* transcriptome assembler we used in the Assembly module.

Our basic output will be a genetic distance matrix, a two-dimensional array (or table) of values ranging from 0 to 1. Each row or column in a genetic distance matrix represents a sample, and the entry in row i, column j quantifies the extent to which the genomes of sample i and sample j differ. Our genetic distance matrices

will be symmetric, meaning that the distance between sample i and sample j is the same as the distance between sample j and sample i. Our genetic distance matrices will also be hollow, meaning that the distance between a sample and itself is always 0.

Although we will be using the main RNA-clique program to perform the full RNA-clique process for us, it is nevertheless helpful to understand the basic steps of RNA-clique for the sake of understanding the parameters we set and the results we obtain. The data produced by the steps of RNA-clique are summarized in the diagram below. Arrows show which pieces of data are used to produce others. The diagram also shows the step of obtaining assembled transcriptomes from RNA-seq reads, which is not part of RNA-clique proper.



RNA-seq reads are used to produce the assembled transcriptomes. Then, RNA-clique selects the top n genes from the transcripts using a metric reported by the assembler called k-mer coverage; this gives us only those genes assembled from much sequence data (often highly expressed genes). RNA-clique uses BLAST to align the top n genes of each sample with those of every other sample and gets pairs of genes between samples that are likely homologs. RNA-clique uses the processed BLAST results to build a "gene matches graph" (a network) representing likely homologous genes and then analyzes the graph to find those genes that appear to have orthologs in every sample. Such genes are part of "ideal components" of the graph. BLAST alignment statistics for genes in ideal components are then used to compute distances for each pair of samples.

This exercise is mainly based on the "From RNA-seq reads to a phylogenetic tree with RNA-clique" document from RNA-clique's manual, and other parts of this exercise also draw heavily from the RNA-clique documentation.

9.2 Inspect the sample metadata

The RNA-seq data used in this exercise are from plants derived from the "Kentucky 31" tall fescue (*Lolium arundinaceum*) cultivar and were originally used in gene expression studies described in (Dinkens et al. 2019). Each set of RNA-seq reads comes from a different individual, and although we will be using transcriptomes assembled from RNA-seq reads for six individuals, the individuals have only four distinct genotypes.

Individuals with the same genotype are clones and should thus have very few variants between their genomes.

Additionally, some of the individuals harbor an endosymbiotic fungus, *Epichloë coenophiala*, while others were treated to remove it. For the purposes of this exercise, each individual (associated with a single set of RNA-Seq reads and a single assembled transcriptome) will be a "sample."

Which samples belong to which genotype, and which have an endophyte? The metadata that answers these questions is present on your VM under the *rnaclique* directory. We've provided metadata in a plain-text comma-separated value format, so we can read them easily with the tools we've learned.

- ☐ Ensure you are in the *rnaseq* directory of your virtual machine.
- ☐ Change into the *rnaclique* directory.

View the *tall_fescue_accs.csv* file to answer the following basic questions:

- ☐ What are the four genotypes of the samples? _____
- ☐ How many endophyte *minus* samples do we have? _____

9.3 Inspect the transcriptomes

Now let's look at the transcriptome assemblies. Each is stored in a directory named after the accession number for the raw data in NCBI's Sequence Read Archive. The transcriptome files themselves are all named *transcripts.fasta* (the default output name given by the rnaSPAdes assembler).

We briefly introduced the rnaSPAdes header format in the Assembly module, but we will revisit it here. Each transcript has a header that beings with `NODE` and ends with `cov_K_gG_iI`, where `K`, `G`, and `I` are numbers. The first number, `K`, is the k-mer coverage of the transcript. The k-mer coverage quantifies the amount of sequence from the input reads that contributes to the assembled transcript. RNA-clique will use k-mer coverage to select genes assembled from many reads.

The second two numbers, `G` and `I`, are the gene and isoform IDs. rnaSPAdes automatically organizes related transcripts into sets called "genes," and each member of a such set of transcripts is referred to as an isoform. These isoforms are generally assumed to represent alternative splice variants of the same gene, though other kinds of related transcripts might also be grouped under the same "gene." Each gene in the assembly is assigned a unique gene ID, and, likewise, each transcript is assigned an isoform ID that is unique within its gene.

Let's examine one of the transcriptome assemblies. Use our command-line tools on the SRR2321385 assembly to answer the following questions:

- ☐ What is the k-mer coverage of the first transcript in the file? _____

We can extract just the gene ID from every FASTA header using **grep** and **sed**. In the grep command below, `.*` means "any sequence of characters."

```
grep -o '_g.*_' SRR2321385/transcripts.fasta | sed 's/_//g'
```

- ☐ After running the command above, are there still multiple lines with the same gene ID? _____
- ☐ How many "genes" are in the assembly? _____

- ☐ What is the maximum number of isoforms that any gene has? (Hint: Isoforms IDs are assigned sequentially starting with 0.) _____

9.4 Activate the RNA-clique conda environment

On your virtual machine, we have many different pieces of software installed. Each piece of software has other software on which it depends; these dependencies must be installed before the dependent software can be used. Those dependencies have their own dependencies, and so on. Managing software dependencies manually can be challenging, so we usually use other pieces of software, called **package managers**, to do this work for us.

Additionally, software dependencies can conflict with each other, so it might not be possible to have software A and software B installed simultaneously. To get around this problem, we can isolate software in their own environments to prevent them from interfering with one another. We activate the environment to use the software and then "deactivate" to restore the original system state. Again, managing environments manually is tricky, so we use special software, **environment managers**, to do it for us.

In this exercise, we will use **conda**, which combines the functions of a package manager and an environment manager to allow installation of software in isolated environments. We have installed RNA-clique in its own **conda** environment, so the first step in using RNA-clique on our VMs is to activate the environment.

- ☐ Enter a **screen** called *rnaclique*.
- ☐ Make sure you are in the *rnaclique* directory.
- ☐ Activate the rna-clique environment:

```
• conda activate rna-clique
```

- ☐ Notice that your prompt has changed. What is different? _____

9.5 Run RNA-clique to get genetic distances

Now that we understand the structure of the input data, we can run RNA-clique on the transcriptomes to calculate genetic distances. RNA-clique doesn't need the sample metadata to compute distances, so we can just provide it the transcriptomes. We also need to tell RNA-clique how many top genes to select based on k-mer coverage. Experiments with this data have shown that selecting 50,000 genes works well.

- ☐ Make sure you are in **screen**, in your *rnaclique* directory, and in the rna-clique environment.
- ☐ Run RNA-clique on the six transcriptomes (all directories under *rnaclique*):

```
• rna-clique -O rna_clique_out -n 50000 */
```

RNA-clique should take about half an hour to finish. Once an RNA-clique analysis has completed, it is usually a good idea to look at the count of ideal components to get an idea of the reliability of the results. More ideal components are better; we typically want counts in the mid-double digits or higher. We can use the RNA-clique **plot_component_sizes** script to get a count of ideal components from a completed RNA-clique analysis.

- ☐ Count the ideal components with **plot_component_sizes**:

```
• python -m rna_clique.plot_component_sizes
         -A rna_clique_out --statistics
```

☐ Examine the output. How many ideal components do we have? Is it enough? (Hint: See the paragraph before the last checkbox.) _____

You'll notice that RNA-clique did not print out a distance matrix when we ran it. To view the computed matrix, we need to use a separate program, **export_matrix**.

☐ Export the distance matrix in table format:

```
• python -m rna_clique.export_matrix -O rna_clique_out -f table
```

You should see a matrix is printed out along with row labels. The labels on the rows indicate to which samples the rows correspond. Although the columns are not labeled, they follow the same order as the rows.

☐ Examine the output matrix. Which pair of samples are the most distant? You may want to pipe the matrix to another program to help you find the answer. _____

9.6 Visualize the distance matrix

The raw distance matrix we obtained in the last section is kind of difficult to interpret, so in this section, we will visualize the same distance matrix to make it easier to understand. The scripts we use to make the visualizations are not part of the RNA-clique suite but use RNA-clique's Application Programming Interface (API) and are based on examples provided in the RNA-clique documentation.

First, let's create a very basic visualization of a distance matrix—a heatmap. In a heatmap, the elements of the distance matrix are represented by rectangles in a grid. Each rectangle t is colored according to a colormap that encodes distances as colors; the colormapwith a scale next to the heatmap. Labels for the rows and columns of the matrix are shown above and to the left of the rows and columns of squares in the heatmap.

☐ Make sure you are in the *rnaclique* directory.

☐ Create a heatmap from the distance matrix:

```
• python make_heatmap.py -O rna_clique_out
                         -M tall_fescue_accs.csv
                         -g genotype -n accession
```

`-O`	RNA-clique output root directory
`-M`	sample metadata CSV location
`-g`	column(s) to group by in plot
`-n`	column providing sample names

☐ Verify that the phylogram has been created at *rna_clique_out/heatmap.svg*.

☐ Transfer the *heatmap.svg* file to your local computer and view it in a web browser.

Notice that in addition to the color mapping, we also have values directly annotated in each cell of the heatmap. These values represent the distances in ten-thousandths. The samples have also been grouped along the vertical and horizontal axes according to genotype.

☐ Do you notice any patterns? Are these patterns what we expected based on what we know about the samples from their metadata? _____

Next, let's create a phylogenetic tree and visualize the tree as a phylogram. The script we use will construct a tree from the distance matrix using the neighbor-joining algorithm and then will draw a diagram in which elements of the tree sharing a parent are joined by horizontal lines with lengths proportional to their distances to the parent. The terminals (leaves) of the tree are the samples in our analysis.

- ☐ Create and visualize a tree from the distance matrix:

```
• python make_tree.py -O rna_clique_out
                      -M tall_fescue_accs.csv
                      -g genotype -n accession
```

- ☐ Verify that the phylogram has been created at *rna_clique_out/tree.svg*.
- ☐ Transfer the *tree.svg* file to your local computer and view it in a web browser.

Finally, let's make a Principal Coordinates Analysis (PCoA, also called multidimensional scaling or MDS) plot. A PCoA plot represents samples as points in a scatter plot. Point coordinates are chosen for the samples such that distances are preserved as well as possible given the number of dimensions allowed.

- ☐ Create a PCoA plot from the distance matrix:

```
• python make_pcoa.py -O rna_clique_out
                      -M tall_fescue_accs.csv
                      -g genotype -n accession
```

- ☐ Verify that the phylogram has been created at *rna_clique_out/pcoa.svg*.
- ☐ Transfer the *pcoa.svg* file to your local computer and view it in a web browser.